HERIOT-WATT UNIVERSITY

HERIOT WATT UNIVERSITY

KNOWLEDGE FOR
A BRIGHTER WORLD

COURSEWORK REPORT

Search and Automated Planning

Sudoku Search Evaluation and Lunar-Rover Modelling

USE OF GENERATIVE AI STATEMENT

Generative AI tools, including OpenAI ChatGPT, were used as supplementary learning aids during this coursework. Their use included preliminary brainstorming and explaining concepts from the lecture slides, particularly constraint satisfaction problems (CSPs), backtracking, heuristic search, STRIPS and PDDL. AI was also used to generate supplementary 9×9 Sudoku test puzzles, with predefined digits and zeros representing empty cells. These puzzles provided additional test inputs for checking the solver's correctness and comparing the performance of different search strategies.

COURSE NAME	F29AI — Artificial Intelligence and Intelligent Agents
GROUP NUMBER	OUC CW 1 25
CAMPUS	Ocean University of China
GROUP MEMBERS & STUDENT IDs	Yingjie Hua — H00490332 / yh4007 Junbiao Huang — H00490384 / jh4007
DATE	28 October 2026

VIDEO DEMONSTRATION GitHub video demonstration (click to open)

COURSEWORK

Part 1. Sudoku search and evaluation

Overview. This report evaluates a Python Sudoku solver based on backtracking, minimum remaining values (MRV), and forward checking. Two simpler variants isolate the effect of each addition. The second part develops a reusable STRIPS model for the lunar-rover missions. Both tasks follow the coursework specification and the search, CSP and planning concepts taught in Weeks 2–4.

1.1 Sudoku as a constraint satisfaction problem (Part 1A)

For each row r and column c, define a variable $X_{r,c}$, where $r, c \in \{1, \ldots, 9\}$. A given digit v fixes its variable's domain to $\{v\}$; an empty cell initially has domain $\{1, \ldots, 9\}$. The file value 0 marks an unassigned cell and is not a candidate digit. A solution satisfies all 27 all-different constraints:

$$\text{AllDifferent}(X_{r,1}, \ldots, X_{r,9}) \quad (r = 1, \ldots, 9), \tag{1}$$

$$\text{AllDifferent}(X_{1,c}, \ldots, X_{9,c}) \quad (c = 1, \ldots, 9), \tag{2}$$

$$\text{AllDifferent}\{X_{r,c} : r \in R_i,\ c \in C_j\} \quad (i, j = 0, 1, 2), \tag{3}$$

where $R_i = C_i = \{3i + 1, 3i + 2, 3i + 3\}$. Given values must also remain unchanged. Equations (1)–(3) can equivalently be expressed as pairwise inequalities between cells sharing a row, column or box. Thus the state describes a partial assignment, and the constraints identify which values may extend it [1].

1.2 Brute force and the effect of pruning

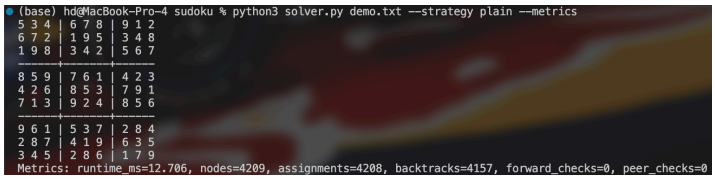
Let m be the number of empty cells. Brute force can enumerate 9^m complete fillings before finding a solution or exhausting the space. With fixed-size 9×9 validity checks, the usual search bound is $O(9^m)$. More explicitly, checking each complete filling adds a validation factor C, giving $O(C9^m)$. Here C is bounded because the grid size is fixed; m is used to compare puzzle instances, rather than to claim a general complexity classification for fixed-size Sudoku.

Backtracking checks consistency after every tentative assignment. With at most nine children and depth m, its worst-case search-tree size is

$$1 + 9 + 9^2 + \cdots + 9^m = \frac{9^{m+1} - 1}{8} = O(9^m). \tag{4}$$

The worst-case exponential bound therefore remains. The practical advantage is that a contradiction at depth k eliminates every completion below that branch, instead of constructing those complete grids first. MRV changes which cell is considered next; forward checking detects some contradictions earlier. Neither guarantees polynomial running time [1].

1.3 Choice of method (Part 1B)



Plain run. Completed grid and counters from the submitted solver.

The implemented method is backtracking with MRV and forward checking. It fits the CSP directly: assigning a digit is simple, a violated constraint is easy to identify, and a failed choice can be undone. MRV exposes restricted cells early, while forward checking rejects an assignment if it leaves a related empty cell without a legal digit. The alternatives in the experiment are controls within the same implementation: `plain` uses row-major selection; `mrv` changes only variable selection; `mrv-fc` also enables the early failure check. Search is implemented explicitly, with no Sudoku or CSP solver library.

Evidence note. The three terminal panels preserve direct runs on the same `demo.txt` puzzle. They demonstrate operation; Table 1 instead uses repeated benchmark medians.

Part 1. Implementation and experimental design

1.4 Representation and recursive search

The board is a list of nine lists. Three collections of sets track digits already used in each row, column and box. For an empty cell, its current candidates are

$$D_{r,c} = \{1, \ldots, 9\} \setminus \big(U_r \cup U_c \cup U_{b(r,c)}\big), \tag{5}$$

where $b(r, c) = 3\lfloor r/3 \rfloor + \lfloor c/3 \rfloor$ for zero-based indices. Updating these sets on assignment and undoing the update on backtracking avoids scanning all units for each candidate test. Candidate digits are tried in ascending order. MRV ties follow row-major order, so completed search counts are reproducible for the same puzzle and strategy.

The main recursion follows the standard backtracking pattern [1]:

1. Select an empty cell. Return success if no empty cells remain.
2. Compute legal values; return failure if the chosen cell has none.
3. Assign one candidate and update the row, column and box sets.
4. In the FC variant, check that each unassigned peer still has a candidate.
5. Recurse if the check passes. On failure, undo the assignment and try the next digit.

Only peers can lose candidates after an assignment, so checking them is sufficient for this forward-checking test. It does not enforce arc consistency between two unassigned cells and does not automatically fill singleton domains. MRV nevertheless selects a singleton as a subsequent search step. This distinction keeps the implementation aligned with the Week 3 techniques.

1.5 Inputs, correctness checks and measurements

The program accepts either a space-separated `.txt` file or a comma-separated `.csv` file. Both formats contain nine rows of nine integers; 0 denotes an empty cell. It rejects wrong dimensions, non-integer values, values outside 0–9, and duplicate non-zero givens. On success it prints nine aligned rows, with separators between the 3×3 regions; for example, the first row of the solved test puzzle is `5 3 4 | 6 7 8 | 9 1 2`. A consistent but unsatisfiable test prints `No solution exists for this puzzle.` A completed solution is checked against all rows, columns and boxes, and the program checks that the givens were preserved.

A node counts an entry into recursive search, including a completed-board call. An assignment counts a candidate placed; a backtrack counts one such placement undone. An FC call counts one invocation of the forward-checking function after a tentative assignment. A peer check counts one unassigned peer whose domain is actually evaluated inside that call; the number varies because checking stops at the first empty domain. A candidate rejected by FC increases assignments and backtracks but need not create a recursive node.

1.6 Fixed evaluation protocol

The test set contains the first five puzzles in Norvig's `easy50`, the first five in `top95`, and the first three in `hardest` [2]. Selection was fixed before the recorded experiment. The labels identify source sets, rather than objective difficulty scores. Each of 13 puzzles was tested with three strategies, one excluded warm-up and five measured repeats: 234 records in total, of which 195 enter the evaluation.

The recorded run used Python 3.11.9 on macOS 15.7.7, ARM64, with a one-second limit per trial [3]. Timing excludes input and printing, but the benchmark's outer timer includes timeout setup and call overhead as well as search. Results are medians in milliseconds. All repeats, including slow observations and timeouts, remain in the raw log. Seven unit tests passed, and separate checks rejected four malformed inputs and correctly identified a structurally valid unsatisfiable puzzle. The solved benchmark groups were also rechecked for valid grids and preserved givens.

Part 1. Results and interpretation

Table 1: Median measured runtimes (ms). TO means all five trials reached the 1,000 ms limit; no completion time is inferred. E, H and X denote the easy50, top95 and hardest source sets.

Puzzle	Givens	Plain	MRV	MRV + FC
E1	32	0.489	0.330	0.655
E2	30	0.650	0.393	0.743
E3	28	46.257	0.901	1.705
E4	30	4.059	0.501	0.770
E5	36	0.182	0.204	0.488
H1	17	TO	7.430	8.579
H2	17	TO	20.340	32.195
H3	17	199.598	19.115	31.611
H4	17	TO	TO	TO
H5	17	TO	298.683	424.600
X1	22	953.932	42.050	69.528
X2	23	26.803	5.205	9.277
X3	26	630.922	1.951	3.026

Plain backtracking completed 9 of 13 puzzles within the limit; both MRV variants completed 12. MRV's benefit was substantial on E3: nodes fell from 19,023 to 131, and the median runtime fell from 46.257 to 0.901 ms. On E5, however, MRV was slightly slower than plain backtracking. A heuristic can reduce work without producing a meaningful speed advantage on an already easy instance.

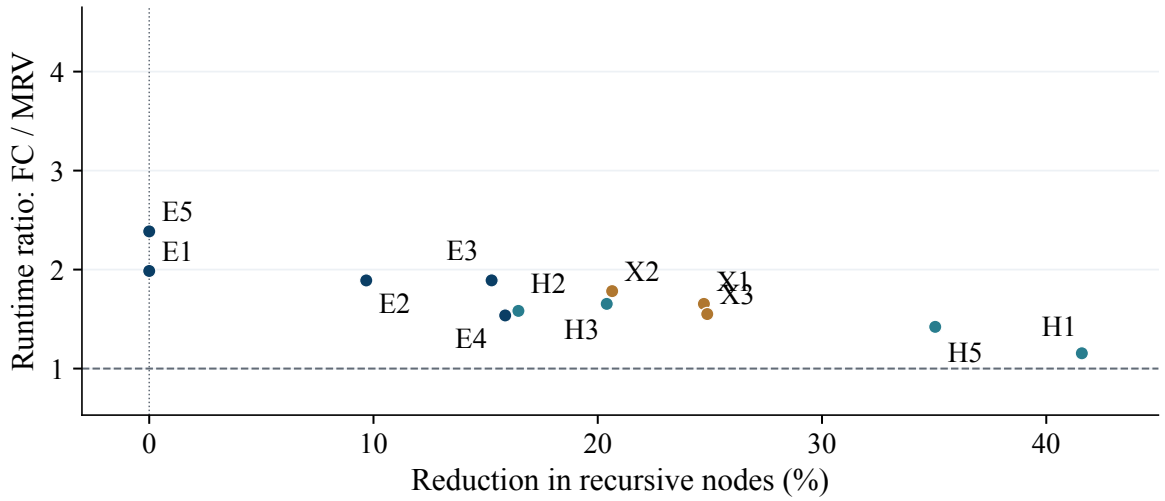
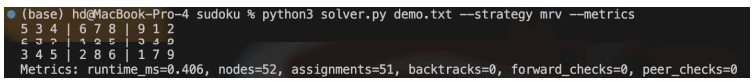


Figure 1: FC trade-off on the 12 jointly solved puzzles. Horizontal position is node reduction relative to MRV; vertical position is the median-runtime ratio FC/MRV. Ratios above 1 indicate slower execution with FC. H4 is excluded because both variants timed out.



MRV run. Direct command and metrics; its printed grid matches the plain and MRV+FC panels.

Figure 1 shows why node count alone is insufficient. FC reduced nodes on 10 of 12 jointly solved cases, but its median time was higher on all 12. On H1, nodes fell from 719 to 420 (41.6%) and backtracks from 654 to 417, yet time rose from 7.430 to 8.579 ms. The 481 FC calls examined 4,094 unassigned peer domains, so repeated scans introduced overhead that competed with pruning.

Part 1. Limitations and theoretical A* analysis

1.7 Failed cases, variability and improvements

H4 reached the one-second limit in every recorded trial, so its baseline counts are partial and excluded from completed-search comparisons. A separate single run per strategy with a 30-second cap then solved H4: plain used 1,007.491 ms and 392,886 nodes; MRV used 1,131.063 ms and 128,756 nodes; MRV+FC used 1,435.819 ms and 76,717 nodes. All produced the same solution hash. These follow-up runs establish solvability, but one run per method is not a stable timing comparison; Table 1 therefore retains TO under its original protocol.

No successful observation was discarded merely for being slow. For example, X1 under plain backtracking ranged from 837.746 to 959.153 ms, while its completed node count stayed fixed. A long time is not an invalid result without evidence of a recording or execution error. Invalid input is handled separately, and the unsatisfiable test is reported as a failure to find a solution after exhaustive search, rather than mixed into solved-puzzle timing statistics.

All five H puzzles have 17 givens, yet their MRV effort ranges from 719 nodes to a timeout. This rules out using clue count alone as a difficulty estimate for this solver. The sample is also small and selected by position in three lists. It supports an implementation comparison, not a claim about all Sudoku puzzles.

The first improvement to test would be incremental domains or cached peer sets: FC currently rebuilds peer sets and recomputes legal digits. A larger fixed dataset, randomised strategy order, and fresh-process trials would help distinguish algorithmic effects from machine load. The long-window follow-up resolved H4, but repeated long-window runs would be needed for a timing estimate. Degree/least-constraining-value ordering or arc consistency are plausible alternatives [1]; their extra bookkeeping must be measured rather than assumed beneficial. The current program has readable terminal output; it does not implement a graphical interface.

```
(base) hd@MacBook-Pro-4 sudoku % python3 solver.py demo.txt --strategy mrv-fc --metrics
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
------+-------+------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
------+-------+------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
Metrics: runtime_ms=0.861, nodes=52, assignments=51, backtracks=0, forward_checks=51, peer_checks=313
```

MRV+FC run. 51 FC calls and 313 peer checks for the same grid.

1.8 Formulating Sudoku as A*

An A* state can be a consistent partial grid containing the givens. An action assigns one legal digit to one empty cell; the goal test requires a complete valid grid. Give every assignment cost 1. If k of the original m blanks have been filled, then $g(s) = k$. A natural heuristic is the number of remaining blanks,

$$h(s) = m - k, \qquad f(s) = g(s) + h(s) = m. \tag{6}$$

For a completable state, exactly $m - k$ assignments remain under this action definition. Thus this heuristic is admissible and consistent: each move decreases h by 1, matching its cost. It also assigns a finite estimate to a dead end, whose true remaining solution cost is infinite. Yet Equation (6) gives the same f value to every reachable depth. It supplies no useful ordering among competing branches; tie-breaking largely determines the search behaviour [1].

Choosing a single next cell, for example by MRV, avoids many duplicate partial grids caused by assigning cells in different orders. Adding dead-end detection or propagation would help A*, but those gains come from CSP reasoning. Every successful filling has the same assignment count m, so shortest-path optimality adds little to this task. A* also stores a frontier of alternative grids, whereas depth-first backtracking keeps a path of at most m assignments plus the board and constraint sets. For these reasons backtracking is a more direct choice here. A* is analysed only; it was not implemented or benchmarked for Sudoku.

Part 2. Lunar-rover planning (Parts 2A and 2B)

2.1 Types, state and reusable actions

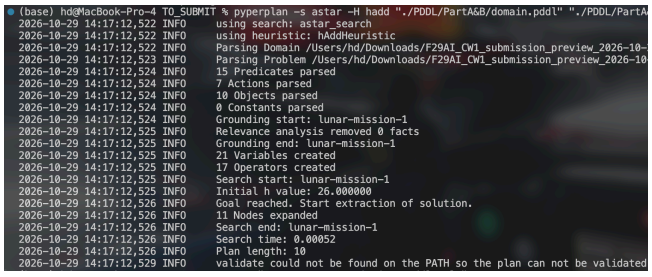
The base domain `lunar` uses only `:strips` and `:typing`. Objects have types `rover`, `lander`, `waypoint`, `sample`, and `data`; `image` and `scan` are subtypes of data. Both data types share a single memory resource. The separation between reusable domain rules and mission-specific objects, facts and goals follows the course planning material [4].

Static predicates include `assigned`, `connected`, `data-at`, and `lander-at`. State predicates distinguish an undeployed rover `aboard` its lander from a `deployed` rover with a `rover-at` fact. Data possession uses `carrying-data`; completed delivery uses `transmitted`. Samples similarly use `sample-at`, `carrying-sample`, and `stored`. A scientific goal refers to the delivery predicate, so acquisition alone cannot satisfy it.

Table 2: Base actions and the modelling decisions that prevent invalid plans. All rover, lander, data, sample and waypoint identities are parameters.

Action	Required conditions and important effects
Deploy	Own lander, rover aboard, lander at w. Add deployed and rover-at w; delete aboard.
Move	Deployed, at source, directed connection. Add destination; delete source location.
Capture image / perform scan	Correct data subtype at rover location and free memory. Add possession; consume memory-free.
Transmit data	Deployed, own lander, data carried. Add transmitted; release memory; delete possession. No co-location condition.
Collect sample	Rover and sample co-located, sample slot free. Add carrying; consume slot; remove sample from site.
Store sample	Own lander, co-location, carried sample and free lander space. Add stored; free rover slot; consume lander space.

Table 2 separates communication from physical transport. Wireless transmission can occur at any waypoint, but storing a sample requires returning to the waypoint of the assigned lander. Landers have fixed initial locations and no relocation action. Deployment is the only way an initially aboard rover obtains its first waypoint.



Mission 1 rerun. Goal reached; 11 nodes; plan length 10.

2.2 Capacity and ownership invariants

Three positive resource predicates avoid numeric fluents: `memory-free` per rover, `sample-space-free` per rover, and `lander-space-free` per lander. Acquisition consumes the relevant free fact; transmission or storage releases the rover resource. Storing a sample consumes the lander's free space permanently because no unloading action exists. Starting from a well-formed state, these transitions preserve capacity one by induction over the plan.

Every deploy, transmit and store action checks `assigned`. Assignment facts never change. Each problem gives every rover exactly one owner and a consistent aboard/deployed state; the domain preserves these relationships rather than repairing malformed initial states. No action hard-codes a rover, lander or waypoint. The same base domain can therefore be used for further valid missions by changing only objects, initial facts and goals.

Part 2. Mission instances and verification

2.3 Initial states, routes and goals

For Mission 1, Lander 1 is fixed at WP1, an allowed choice because the landing site is unspecified. Rover 1 starts aboard. The directed edges are WP5→WP1, WP1→WP2, WP2→WP3, WP3→WP5, WP1→WP4, and WP4→WP3. The goals require the WP5 image and WP3 scan to be transmitted to Lander 1, and the WP1 sample to be stored there. The saved 10-action plan deploys, collects/stores the local sample, follows WP1–WP4–WP3–WP5, and acquires/transmits both data items.

For Mission 2, Rover 1 is deployed at WP2 beside Lander 1. Lander 2 is chosen to be at WP1 and Rover 2 starts aboard it. Bidirectional pairs are WP1↔WP2, WP2↔WP4, and WP3↔WP5. Additional one-way edges are WP2→WP3, WP5→WP6, and WP6→WP4. Thus a return from the WP3/WP5 area can use WP5–WP6–WP4–WP2; there is no invented WP3→WP2 edge.

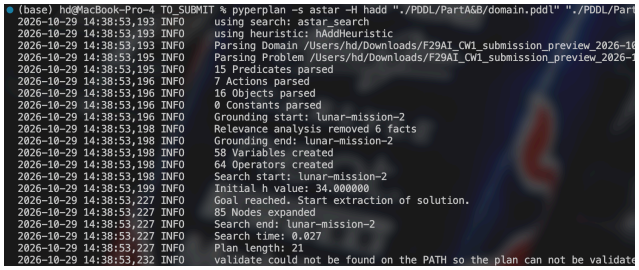
Ownership assigns the WP3 image, WP6 scan and WP5 sample to Rover 1/Lander 1, while Rover 2/Lander 2 handles the WP2 image, WP4 scan and WP1 sample. The six goals name the destination lander explicitly. Mission 3 keeps these external facts and goals and adds astronaut state.

2.4 Positive plans and negative tests

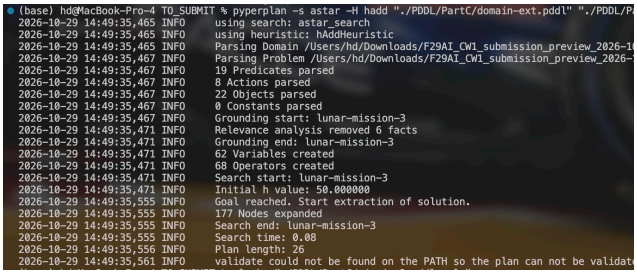
Pyperplan 2.1 was run with `-s astar -H hadd`. The additive heuristic can overestimate, so these are feasible plans, not certified shortest plans. Table 3 records the preserved planner output. Each saved plan was replayed from the grounded initial state: every action's preconditions held and all goal facts were present at the end. This replay uses Pyperplan's semantics; a separate VAL validator was unavailable [5].

Table 3: Recorded planning results. Expanded nodes and plan lengths belong to one saved run and may change with tie-breaking.

Mission	Actions	Expanded nodes	Goal facts reached
1	10	11	3/3
2	21	85	6/6
3	26	178	6/6



Mission 2 rerun. Goal reached; 85 nodes; plan length 21.



Mission 3 rerun. Goal reached; 177 nodes; plan length 26. Table 3 records 178 nodes from the preserved run, showing tie-breaking variability.

Six small, deliberately impossible problems also returned no solution: transmission without any acquisition source; transmission to a different lander; sample storage after a one-way trip with no return edge; simultaneous possession of two data items; two samples stored in a capacity-one lander; and deployment when the astronaut is isolated in the control room. These test prerequisites and resources directly, rather than relying only on a successful mission plan.

An unsolvable test alone is not a proof of general correctness: a missing positive action could also make a problem unsolvable. The evidence therefore combines solvable missions, action replay, negative cases, and the resource/ownership argument in Section 2.2. The additional test problems remain supporting material; the submitted PDDL set contains exactly the starter's two domain and three mission files.

Part 2. Astronaut extension and conclusions (Part 2C)

2.5 Astronaut state that restricts action applicability

The extended domain `lunar-extended` adds types `astronaut` and `area`, with `control-room` and `docking-bay` as area subtypes. Static `astronaut-assigned` and `area-in` facts associate an astronaut and both rooms with one lander. `astronaut-at` records the current room. A move requires source and destination to belong to the astronaut's own lander and a directed `room-connected` fact; its effects replace the old location with the new one.

Deployment and sample storage additionally require the assigned astronaut in that lander's docking bay. Transmission requires the assigned astronaut in its control room. Thus the room facts change which actions are applicable. The rover's movement, acquisition and capacity rules are inherited from the base model. Alice initially occupies Lander 1's docking bay and Bob Lander 2's control room, exactly as required.

2.6 How the extension changes the Mission 3 plan

Table 4 extracts dependencies from the saved 26-action plan. The listed actions are in plan order; intervening rover actions are omitted for clarity. Initially, Bob cannot deploy Rover 2 from the control room, and Alice cannot support a transmission from the docking bay. Both must move before those actions can occur.

Table 4: Astronaut movements and the dependent operations in the recorded Mission 3 plan. Numbers are positions in the full plan.

Step	Astronaut movement	Operation enabled
1	Bob: control room → docking bay	Deploy Rover 2 (2); store WP1 sample (5).
3	Alice: docking bay → control room	Support Rover 1 transmissions, first at step 20.
6	Bob: docking bay → control room	Transmit WP2 image (9) and WP4 scan (12).
21	Alice: control room → docking bay	Store WP5 sample (22).
23	Alice: docking bay → control room	Transmit WP3 image (26).

Mission 2 uses 21 actions and Mission 3 uses 26 in this run. Removing the five astronaut moves from the Mission 3 plan leaves the same 21 rover operations in the same order as the saved Mission 2 plan. The observed increase is therefore directly attributable to astronaut coordination for this pair of plans, rather than a change in the mission goals or map.

Five astronaut moves are not proven necessary. Rover 1 could capture/transmit the WP3 image on its first visit before collecting the WP5 sample, potentially avoiding Alice's second return to the control room. The saved planner chose a different order. This illustrates why plan length alone cannot establish modelling quality or optimality: the key requirement is that every gate is enforced and every goal is reached.

2.7 Overall conclusion

The Sudoku evaluation supports MRV as a useful ordering heuristic on this fixed set: it increased within-limit coverage from 9/13 to 12/13. FC pruned additional nodes on most solved cases, but its runtime overhead outweighed that saving here. H4 is solvable, although all baseline trials were censored by the one-second limit. The next useful experiment is an incremental-domain FC implementation with the same fixed inputs and a less noisy timing protocol.

The planning models represent directed movement, wireless data delivery, physical sample return, ownership and capacity using course-level STRIPS features. All three saved plans reach their goals under action replay, and the astronaut extension visibly restricts deployment, storage and transmission. Further tests should vary valid maps and starting states and check each astronaut gate separately; the current evidence establishes the required examples and selected failure cases, not every possible instance.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021, chs. 3, 6, and 11. [Online]. Available: `https://aima.cs.berkeley.edu/`

[2] P. Norvig, "Solving Every Sudoku Puzzle," 2006. Benchmark data: `easy50.txt`, `top95.txt`, and `hardest.txt`. [Online]. Available: `https://norvig.com/sudoku.html`. Accessed: Oct. 28, 2026.

[3] "F29AI CW1 experimental record," reproducibility files, run dated Oct. 28, 2026. Sudoku: benchmark inputs and manifest, `raw_runs.csv`, `summary_by_puzzle.csv`, `environment.json`, and `h4_extended.json`. Planning: saved Mission 1–3 Pyperplan logs, `.soln` plans, and plan-replay results.

[4] M. Najib, "F29AI: PDDL practical tutorials," Heriot-Watt University. [Online]. Available: `https://github.com/valvestate/f29ai/tree/main/pddl-tutorials`. Accessed: Oct. 28, 2026.

[5] Pyperplan contributors, "Pyperplan: A lightweight STRIPS planner written in Python," software and documentation. Recorded environment: version 2.1. [Online]. Available: `https://github.com/aibasel/pyperplan`. Accessed: Oct. 28, 2026.